

Half-Baked Cookies: Hardening Cookie-Based Authentication for the Modern Web

Yogesh Mundada
Georgia Tech

Nick Feamster
Princeton University

Balachander Krishnamurthy
AT&T Labs – Research

Abstract

Modern websites use multiple authentication cookies to allow visitors to the site different levels of access. The complexity of modern web applications can make it difficult for a web application programmer to ensure that the use of authentication cookies does not introduce vulnerabilities. Even when a programmer has access to all of the source code, this analysis can be challenging; the problem becomes even more vexing when web programmers cobble together off-the-shelf libraries to implement authentication. We have assembled a checklist for modern web programmers to verify that the cookie based authentication mechanism is securely implemented. Then, we developed a tool, *Newton*, to help a web application programmer to identify authentication cookies for specific parts of the website and to verify that they are securely implemented according to the checklist. We used *Newton* to analyze 149 sites, including the Alexa top-200 and many other popular sites across a range of categories including search, shopping, and finance. We found that 113 of them—including high-profile sites such as Yahoo, Amazon, and Fidelity—were vulnerable to hijacking attacks. Many websites have already acknowledged and fixed the vulnerabilities that we found using *Newton* and reported to them.

1 Introduction

The web’s core protocol, the Hypertext Transfer Protocol (HTTP), is inherently stateless. To manage higher-level application states, web applications commonly store information about user sessions in “cookies.” In addition to maintaining the user’s state, websites also use cookies to *authenticate* users. A user who initially logs into a website using a username and password could subsequently simply present *authentication cookie(s)* (*auth-cookies*) for access to that site. Once a user authenticates to a web server, the auth-cookies are thus a critical security linchpin: In many cases, access to these auth-cookies gives an attacker complete control over a user’s account [28].

Modern web applications are complex. They have millions of lines of code, and often use legacy code from other components or libraries. Today’s web sites and applications are multi-faceted, so a “login” or “authentication” is no longer strictly binary. For example, a user may have different levels of authorization or access to different parts of a web site (*e.g.*, a user can view account balances but not execute trades, or add items to a shopping cart but not purchase items or ship to a different address). For such complex applications,

ensuring that authentication logic is correct can be a challenging task—one that, as we have discovered, web developers often get wrong. For example, in May 2014, the incorrect security settings of WordPress’s auth-cookies left users vulnerable to session hijacking attacks [2].

Our results show that these problems are not isolated: We found 113 different sites—including 62 Alexa top-200 sites—that were vulnerable to attacks where an attacker could steal auth-cookies to gain unauthorized access to parts of a site or its entirety. (In 10 cases, we were able to find a way to report the vulnerability to the site operators and did so.) We also evaluated popular eCommerce frameworks—Magento [17], BigCommerce [5], WooCommerce [33], and Volusion [31]—which are used in building millions of eCommerce sites; more than 900,000 sites that use these off-the-shelf frameworks may still be vulnerable to session hijacking attacks. In some of those cases, such as Vimeo and BigCommerce, the developers have confirmed and fixed the bugs that we have found; Amazon is in the process of fixing the bug that we reported to them.

Finding these vulnerabilities highlights an important and prevalent weakness in today’s web authentication frameworks. Finding the vulnerabilities was a challenging problem in its own right, since we generally cannot assume access to the source code of many web applications. Indeed, even a developer who has complete access to source code may find the analysis challenging, due to the complex nature of these applications. Thus, we needed to design a black-box approach to detect authorization cookies for a web application. We developed a tool called *Newton* that developers can use to help them more easily identify these vulnerabilities. We present the following contributions:

- We suggest improvements to existing best practices for securely implementing auth-cookies. We assembled a 6 rule checklist for secure auth-cookies implementation by making 5 of the existing rules precise, while adding one more of our own.
- We developed a tool, *Newton*, an easy-to-use Chrome extension. Anyone can use *Newton* to identify the auth-cookies required to access a portion of a website and analyze if these auth-cookies are sufficiently protected using the security checklist.
- Using *Newton*, we conducted a security audit of 149 sites, with the help of our pilot users, and found previously unknown vulnerabilities in 113 sites, including Yahoo, Vimeo, and BigCommerce. We have reported these problems, and many of them have either fixed already or acknowledged the existing problem (which sometimes has been attributed to a third-party library).

2 Modern Web Authentication

We present general background on web cookie operation, introduce a model for how *auth-cookies* authenticate users on websites to-

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or re-publish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

AsiaCCS '16, May 30–June 3, 2016, Xi'an, China

Copyright 2016 ACM. ISBN 978-1-4503-4233-9/16/05 ...\$15.00

<http://dx.doi.org/10.1145/2897845.2897889>

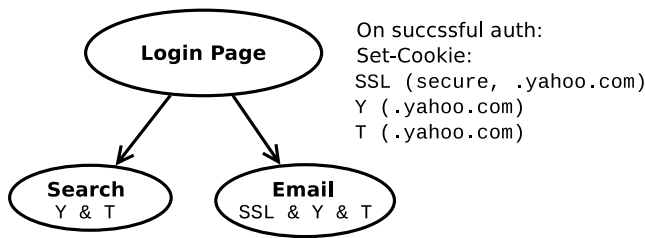


Figure 1: A simplified authentication model for yahoo.com.

day, and describe scenarios where these cookies might be stolen or otherwise compromised.

2.1 Formal Authentication Model

When a user visits a website, the web server sets *cookie* properties (i.e., name, value, flags, expiration date, and a match rule) either statically using `Set-Cookie` or with JavaScript. When a user makes an HTTP or HTTPS request, the browser sends cookies whose match rules correspond to the requested URL and protocol. A browser will not transmit cookies with the HTTPS attribute over a regular HTTP connection.

A web server sets multiple *auth-cookies* when a user initially authenticates to the web server. Auth-cookies ensure that a user's session remains continuous in case connectivity is interrupted (e.g., due to termination of a TCP session or a change in the user's IP address). Many websites use different combinations of auth-cookies to control access to different parts of the site (e.g., a user filling a shopping cart might require a different set of cookies than purchasing the items in the cart).

We can formally represent a user's logged-in state to any part of a site as a disjunctive normal form (DNF) formula over cookies. Any one of the multiple conjunctions of auth-cookies (each with the correct value that matches exactly) can allow a user to access that part of a site. For example, to access dating site "match.com," the browser must satisfy DNF: (Handle AND Password) OR (SECURE). Note that is not a generic version of DNF, because every site requires at least one auth-cookie to authenticate a user.

We call each "one or more conjunctions of one or more literals" a DNF term. Calzavara *et al.* refer to them as authentication tokens [8]. Whereas the related work only focuses on authentication cookies used to login to the main part of the site, we discovered that often sites have multiple parts controlled by different DNFs. A user's logged-in state on each part of the site is decided by relevant DNF for that part. Furthermore, a site may have login dependencies, such that logging into (or out of) one part of a site results in automatic login (or logout) to other parts of the site. We can represent those dependencies as a directed graph, where a directed edge from *A* to *B* means that the login state of *B* depends on the login state of *A*.

We aim to discover (1) the DNF formula for authenticating a user to each independent part of a website; (2) based on the DNF formula, the dependencies between each part of the site, if any exist. Once we have recovered this structure, we can determine whether any combination of cookies could be stolen by an attacker, thus resulting in unauthorized access. (Section 2.2 discusses our threat model for an attacker.)

Example Suppose that when a user logs into yahoo.com, the site sets the Y, T, and SSL cookies. The Y, T, and SSL cookies match on the domain .yahoo.com and path("/"), but the SSL cookie only matches on HTTPS connections to .yahoo.com. Different combinations of cookies (and their correct values) authenticate the

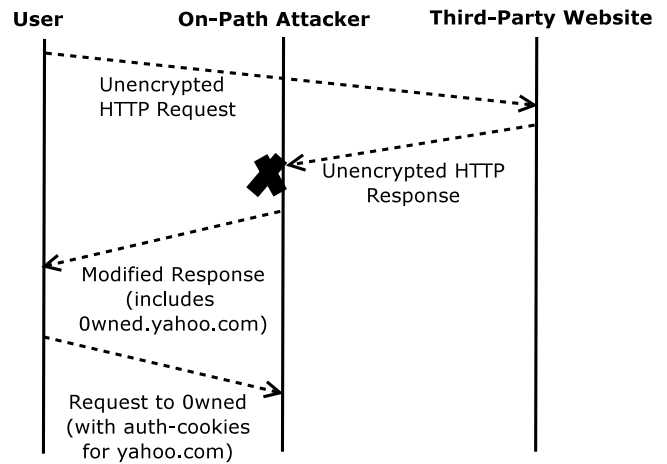


Figure 2: An on-path attacker can gain unauthorized access to a user's search history. (We discovered this vulnerability on yahoo.com.)

user to different parts of the site: Y and T allow a user to access search history; accessing email requires the SSL cookie, as well.

Figure 1 formally represents the web authentication model for this simplified example of the login process on yahoo.com. A successful login automatically results in login to the search and email portions of the website (and the corresponding cookies being set). Each part of the website has a corresponding DNF login formula, as shown within the node. If a user logs out of the "login" portion of the site, the user is logged out of both the search and email portions of the site, as represented by the dependency graph. However, a user can be logged out of email part yet remain logged in on the search portion of the site.

2.2 Threat Model

As discussed earlier, once a user is logged in, auth-cookies become a security linchpin. If an attacker can steal these cookies, then she can hijack the user's session. In this section, we list a couple of different ways in which a victim's auth-cookies can be harvested.

First, we are concerned with the threat of an on-path attacker who intercepts unencrypted auth-cookies. In this case, a vulnerable DNS server would allow an attacker to poison its cache and reroute a victim's traffic to machines under the attacker's control [3]. An on-path attacker or any attacker who can control and manipulate DNS traffic destined for a user's machine—as is the case with untrusted WiFi hotspots, DNS servers that are vulnerable to cache poisoning, or rogue ISPs—can induce a user's browser to submit all auth-cookies without the `secure` flag across any site where the user is logged in.

Figure 2 illustrates this attack. In this case, the attacker is the administrator of the hotspot or has otherwise compromised it, perhaps using off-the-shelf tools [12, 18]. Suppose that the victim is logged into a Yahoo account and always accesses search.yahoo.com over HTTPS. The two cookies that allow access to a user's search history, Y and T, do not have the `secure` flag set. Thus, if any other HTTP webpage that an attacker controls injects content from a yahoo.com subdomain, the browser will leak the auth-cookies to the attacker's machine.

Second, if a website has other vulnerabilities, such as cross-site scripting (a common vulnerability [21]), an attacker may also be able to steal a user's auth-cookies that are not protected with the `HttpOnly` flag. Third, a user may log into a site from an untrusted device. Then even if the user logs out of the site, if the site does

not invalidate auth-cookies, then the administrator of that untrusted device could take over the user's session. Fourth, a user's personal device may be stolen. If the user is already logged into a couple of different services in a browser on that device, and the site does not implement auth-cookies security correctly, then the sessions may remain valid, even if the user changes his or her password.

2.3 Limitations of Existing Defenses

We now will explore existing defenses against session hijacking via cookie stealing, as well as the limitations of these defenses.

Cookie flags To protect cookies against theft by untrusted third parties, each cookie also has two boolean attributes, `HttpOnly` and `Secure`. Javascript running in the user's browser can read, write, and delete cookies created by the same domain [4]. If the cookie's `HttpOnly` flag is set to true, then even JavaScript from the same domain cannot read that cookie, which thwarts potential cross-site scripting attacks [14]. If a cookie's `Secure` attribute is set, then the browser will only return the cookie over HTTPS.

One might think that perhaps all cookies involving authorization should have the `secure` and `HttpOnly` attribute set, but code complexity can make it difficult for a developer to identify the set of auth-cookies that are used for some portion of a website. Additionally, setting `HttpOnly` on all cookies is not practical because some JavaScript functions legitimately require access to these cookies. One example is "double submit cookies" [23], a mechanism to prevent cross-site request forgery whereby a web browser sends a cookie with some value in the HTTP header, and the JavaScript code sends the same value in the HTTP request body. This mechanism requires the JavaScript to access a cookie value, so setting `HttpOnly` is not possible. Setting `secure` on all auth-cookies is not practical, either, because an application may serve content over both HTTP and HTTPS.

HTTP Strict Transport Security (HSTS). HTTP Strict Transport Security (HSTS) defends against certain man-in-the-middle attacks by forcing some web requests to upgrade from HTTP to HTTPS. If the server sets the HSTS policy such that an auth-cookie-domain is a subdomain of the HSTS policy, then the browser would always send the auth-cookie over HTTPS, even if the `secure` flag is not set on the cookie. HSTS could overcome situations where a developer fails to set the `secure` flag on an auth-cookie, but unfortunately this approach does not always work.

Deploying HSTS requires careful analysis of a site, and even reorganizing website content into different subdomains. In some cases, reorganizing a site to use HSTS without breaking the site's functionality is not even possible [15]. HSTS is still not widely deployed. According to a report in November 2014, only 3,406 sites out of approximately 150,000 popular sites had deployed HSTS [29]. Even if HSTS is deployed, it may be vulnerable to SSL downgrading attacks [13] and is often misconfigured [16].

3 Newton: A General Cookie Auditing Tool

To check whether the best practices from the previous section are followed by a website, one has to first identify auth-cookies and their exact DNF required to access a particular part of the site. Unlike Calzavara *et al.* who run their algorithm in a controlled lab setting, we aimed towards four different kinds of users: a) application developers performing black-box testing, b) penetration testers without source-code access identify vulnerabilities in existing web applications, c) security-conscious users identify when they might be vulnerable to session hijacking attacks, and d) as a crowd sourcing

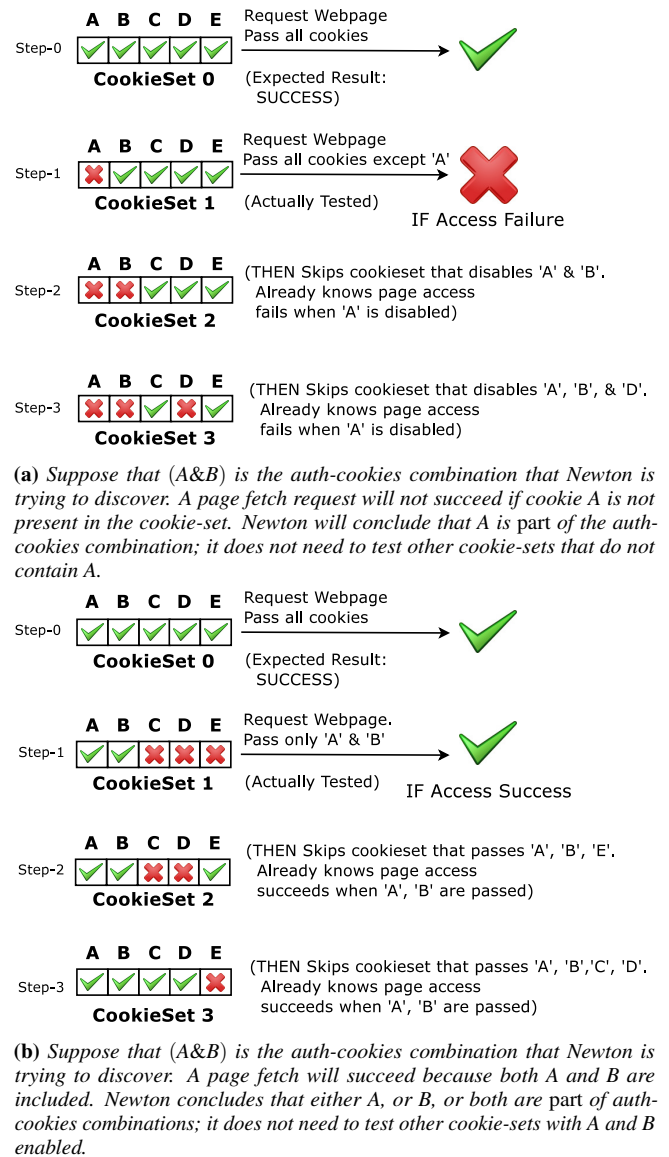


Figure 3: Optimizations.

platform where users can run the tool and find DNFs on different sites with minimal configuration.

In this section, we first list the similarities and differences of our algorithm from previous work. Then we list the specific goals that are different due to our decision of running the tool in non-controlled setting.

3.1 Basic Algorithm

Newton identifies a website's auth-cookies using the following approach:

1. Retrieve a webpage without suppressing any cookies,
2. Disable a combination of cookies and then retrieve a webpage with rest of the cookies enabled.
3. Compare the response page with the one fetched in Step 1.
4. If the webpage is not retrieved successfully, then we mark the combination of disabled cookies as required for accessing the webpage.

This approach faces two major challenges: (1) there are exponentially many possible combinations of cookies to test; (2) the unique design of each website makes it difficult to develop tests and draw conclusions about success or failure—even determining whether a user is logged in can be surprisingly challenging, for example.

3.2 Challenges

In this section, we elaborate on the two major challenges presented above as well as how Newton addresses them.

3.2.1 Efficiently Testing Cookies

At first glance, it might appear that if a website sets N “login cookies,” then Newton would need to send HTTP requests to the website with all 2^N possible cookie combinations to determine all sets of cookies that represent auth-cookies. Fortunately, we can use the outcome of some cookie-set tests to infer the outcomes for other cookie-sets, precluding the need to test all sets with HTTP requests. Figure 3a shows one such inference optimization: if Newton determines that the user is logged in when all cookies in the cookie-set are sent, but that the user is logged out when cookie A is not set, then Newton can conclude that A is *part* of an auth-cookies combination. Conversely, if Newton determines that a user is logged in when all cookies in a cookie-set are sent, and also that the user is logged in when *only* cookies A and B are sent, then Newton can infer that any cookie-set containing A and B will succeed, and either A or B or both of them are auth-cookies. Figure 3b shows this optimization. These optimizations are similar to pruning techniques used by Calzavara *et al.*. Their DNF terms generation mechanism is faster than ours due to using the Apriori algorithm [1]. In spite of using suboptimal DNF term generation, our auth-cookie computations typically finish within five minutes. Incorporating the Apriori algorithm into Newton is a straightforward extension.

3.2.2 Running Newton in Non-Controlled Settings

Instead of guessing auth-cookies for the main part of the site—as is the state of the art in existing approaches—it is imperative to compute the *exact* DNF required for critical parts of a site. Interestingly, this task actually is easier if we collect data from when a user is naturally browsing the web, rather than in a controlled setting. The main idea is that Newton automatically detects the DNFs across multiple parts of multiple sites and compiles this information at a central place for various uses. This design makes it much easier to gather more data at scale, but requires Newton to run in the background on real users’ machines, which requires solving the challenges that we describe in the rest of the section.

3.2.3 Deciding Which Portions of a Site to Test

To exhaustively enumerate all DNFs for a website, the tool needs to know all critical parts of the site. This task is easier when a site developer or penetration tester explicitly identifies different logical parts. Yet, if this task is performed by a crowd sourcing platform, the tool must automatically identify critical parts of the site and execute tests on each portion of the site independently. In such cases, the tool identifies whether the user has entered a password and logged in to the site recently. The tool may also need to apply some heuristics, such as detecting that some portion of the site is served over HTTPS, or the presence of critical information such as credit card numbers or social security numbers.

3.2.4 Detecting Login Success or Failure

Identifying login cookies also depends on an accurate mechanism for determining whether the user is logged in, yet differences in

website design can make even this seemingly simple task surprisingly difficult. Like Calzavara *et al.*, we found that a reasonable heuristic for doing so is to determine the presence of a username on the site itself. However, since we wanted our tool to run as a crowd sourcing platform with minimal configuration, we did not want ordinary users to enter their usernames into the tool. For this reason, we wrote scrapers in the tool to automatically download and populate username corpus from popular 70 sites such as Google, Facebook, Yahoo, etc. In our experience, in most cases the usernames from these sites are sufficient to identify usernames on unknown sites. Section-5 provides an example of one such scraper for dropbox.

3.2.5 Disambiguating Authentication Failures

Finding the correct DNF of auth-cookies depends on correctly identifying whether the user is logged in on the currently loaded page. Unfortunately, various transient conditions can also introduce spurious failures that exhibit similar characteristics to login failures, and we must be able to disambiguate these from actual login failures. For example, a user might log out of a site, or the user’s session might time out. To disambiguate these results, Newton must also reload pages with all cookies enabled after getting a positive result to verify that the user is still logged in. If Newton detects that the user has purposefully logged out of a site, it will store the current DNF testing state for that part of the site and resume the computation the next time user accesses that part. Sometimes an authentication failure may occur because of poor network conditions. Newton waits a sufficiently long time for the page to load and attempts reload under poor conditions, to minimize the likelihood of mistaking poor connectivity for a login failure. Newton determines the waiting-time threshold by maintaining a moving average of past successful page load times.

3.2.6 Avoiding Disruptions to User Session

Sending a website incorrect cookies can cause the website to log a user out, disrupting the user session. For example, Facebook has an auth-cookies combination ‘ xs AND c_user ’, and dropping xs resets the c_user cookie and logs the user out, thus disrupting the user experience. We needed to devise a mechanism to send arbitrary cookie sets from a real user’s browser *without disrupting the user if we send an incorrect or invalid set of cookies*. To do so, Newton runs all tests using an in-memory set of shadow cookies that mimic the user’s real set of cookies.

Because Newton repeatedly loads the same page during auth-cookies computation, one concern is that it could affect user’s state on the server side. A page load is an HTTP GET request which is an idempotent operation [10]. It should only have an effect the first time the user performs the request. It is possible that repeated requests might have unintended side-effects, however (e.g., a user might be rate-limited if the client is perceived to be performing excessive requests). For these reasons, we are cautiously limiting our real world study with the alpha users. Although so far, we have not seen any side effects either in our lab testing or from our users.

Some of Newton’s tests of a website’s practices depend on interactions with the user that should be as infrequent as possible or are not guaranteed to occur in the first place. For example, to test that a site invalidates a user’s auth-cookies after the user has logged out, Newton must discover that the user has logged out, restore the previous session cookies, and test if the user is still logged in. Newton automatically discovers when a user logs out using various heuristics, but to restore the auth-cookies, we explicitly ask for user permission, which becomes annoying if performed repeatedly. Testing whether

a website changes the user’s auth-cookies when the user changes his or her password requires waiting for the user to first change his or her password. Currently, Newton automatically discovers when a user changes his or her password for a particular account, at which point it subsequently performs testing. Because Newton must wait until a user changes his or her password to perform this test, we cannot guarantee that this test will always be done.

Because Newton retrieves webpages repeatedly while suppressing different combinations of cookies, a concern is that such repeated requests might have unintended or unforeseen side effects. For example, some sites aggressively monitor auth-cookies in the HTTP requests, and if the HTTP request sends fewer auth-cookies than required for successful authentication, the server may log the user out. While this action reflects good security practice, it also makes it more difficult to compute auth-cookies. In the future, maintaining instances of such interference or making auth-cookie computation probabilistic could help mitigate some of these effects.

4 Newton: Computing Auth-Cookies

In this section, we describe the design of Newton, a tool that we have built to help website administrators in auditing, and to help common users to protect their web sessions. We refine the basic idea that we presented in Section 3.1 to address the various practical concerns that we raised in Section 3.2.

4.1 Detecting Login Status: Username Presence

Newton uses the presence of the user’s name on the webpage to determine that the user is logged in. To detect the presence of a user’s name without explicitly asking the user (which is error-prone in any case, since the webpage may display a real name to a logged-in user, rather than the username), we developed a *domain specific language* that allows us to specify how to scrape a user’s username and other information from different sites. Using this language, we developed scraping scripts for many popular sites, including Facebook, Google+, Amazon, and many dating sites to determine a user’s real name and username.

Determining a user’s login status on a broad range of sites using the presence of a username or real name on the page requires multiple bootstrapping steps. First, we create dummy accounts on an initial “seed” set of sites to help us codify the relationship between the site’s structure and the location on the site where the user’s name resides. Knowledge of these relationships allows us to develop scripts in our domain specific language to scrape real names and usernames from these sites for any user who installs Newton. We then ran Newton for each webpage for each of the sites where we could determine these relationships. Newton then determines the auth-cookies for each of these webpages (using the algorithms that we describe below). After the user installs Newton, the tool can determine whether a user is logged in on these sites by examining whether the appropriate auth-cookies are set; it can then determine the user’s name (or username, depending on the site) by scraping the page. Finally, with knowledge of the user’s name(s) and username(s) from these seed sites, Newton can then determine whether a user is likely logged in on other sites (e.g., banking sites) simply by checking for the user’s name (or username).

Using our domain specific language, we have created 70 scraping scripts for top sites where users have accounts usually such as social networking, mailing, utility sites. Using these, we are able to comprehensively cover different variations in a specific user’s usernames and thus detect these usernames on sites, even on sites for which we have not written a scraper. However, even if we fail to detect

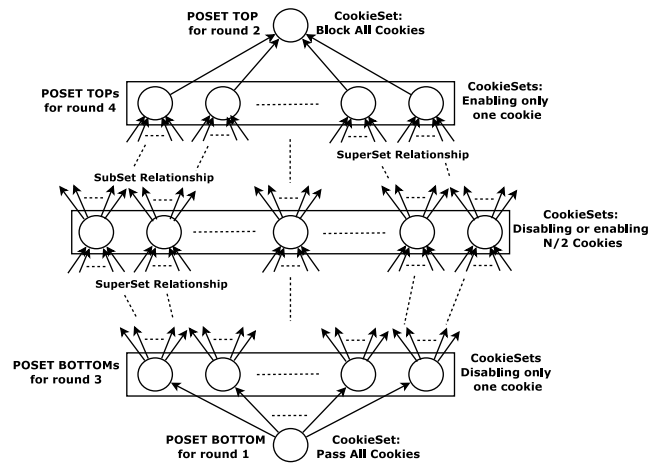


Figure 4: Bounded complete partially ordered set (POSET) representing the set of all cookie-set combinations that Newton would have to test without optimizations. In each testing round, Newton alternately tests current “tops” or “bottoms” of the POSET.

some user’s username on some site, Newton can *still* determine the auth-cookie combination for that site by performing the auth-cookie computation for a user who we are able to identify for that site. Because the auth-cookies that a site uses are the same for every user, Newton can build a knowledge base of auth-cookies for a large set of sites that is then shared across Newton users.

4.2 Tackling Combinatorial Explosion

The strawman algorithm in Section 3.1 computes auth-cookies by disabling combinations of cookies and determining whether a user remains logged in. Although this approach is simple and effective, it does not scale in practice, because the number of tests is exponential in the number of cookies that a site sets, and many sites set tens or hundreds of cookies; once the number of cookies on a site exceeds about 30, testing every combination becomes prohibitive. For example, `live.com` sets 27 cookies at login time. Assuming a single test requires about 200 ms, testing all combinations of only those cookies would require nearly a year. In this section, we discuss how to avoid the exponential cost of testing every possible cookie combination. Although the search space remains exponential, Newton can use knowledge from previous results to avoid testing certain combinations of cookies.

4.2.1 Basic optimization

First, Newton partitions the cookies for a site into two sets: “login cookies” (set during a user’s login process) and “non-login cookies” (set either before or after the login event). To reduce testing time where possible, Newton initially assumes that a site’s auth-cookies are a subset of the cookies that are set at login (the “login cookies”). As we discussed in Section 3.2, some auth-cookies may not be set at login, and Newton may need to expand the set of cookie-sets that it is testing to include additional cookies.

Figure 4 illustrates bounded partially ordered set representing all possible cookie-sets that Newton would have to test, either working downward from the top of the graph by adding cookies to an empty cookie-set, or by working upward from the bottom by removing cookies from a complete cookie-set. The Newton algorithm alternates by performing one test from the top of the partially ordered set, followed by one from the bottom, removing cookie-sets from

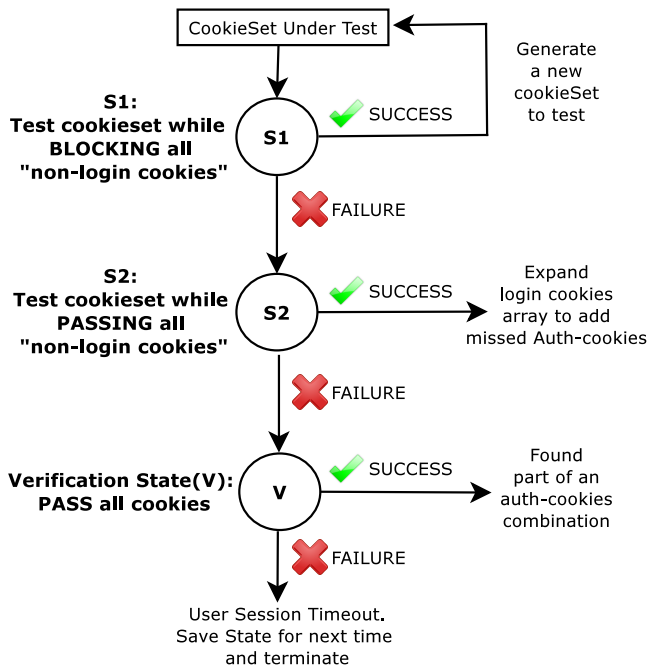


Figure 5: When testing all current bottom cookie-sets (going UP in the POSET), if Newton discovers a negative test result (testing a cookie-set still causes the user to be logged in), then Newton moves on to the next cookie-set to be tested. However, if the result is positive, then Newton performs two additional tests: one to ensure that there are no additional auth-cookies that are in non-login cookie set (state S2), and one to ensure that we have not mistakenly assumed a disabled cookies as an auth-cookie combination because a user was logged out during testing (state V).

the graph as they are determined to either represent an auth-cookie set or not. The algorithm terminates when no cookie-sets remain.

4.2.2 Handling corner cases

User logout or session termination during test. When Newton is computing the auth-cookies for a particular site, the user's session may terminate for a variety of independent reasons; a user's session might time out, or a user might initiate a logout. If these events occur during one of Newton's tests, logout should not be attributed to the set of cookies that were being tested at the time the logout occurred. For this reason, after the computation of the auth-cookies completes, Newton tests that the user is still logged in by sending all cookies for the site in an HTTP request. State V in Figures 5 and 6 illustrates this logic.

Finding auth-cookies that are not set at login. As we mentioned in Section 4.2.1, sometimes an auth-cookie might be set at some time other than when the user logs in, either because the site sets it at some later point after the user logs in (e.g., as is the case with Google Calendar) or because the auth-cookie persisted since the user's previous session. There are two ways to detect that auth-cookies may be set at times other than when the user logs in; states S1 and S2 in Figures 5 and 6 show the logic to detect these corner cases. For example, in Figure 5, if a set of cookies from the "login cookies" set does not log a user in, but enabling all of the cookies that are not set at login results in a successful login, then Newton detects that one or more cookies that were not set at login must be part of the set of auth-cookies. In this case, Newton must actually proceed to compute the exact auth-cookies present in the "non-login cookies" set. To do so, Newton constructs a partially ordered set of

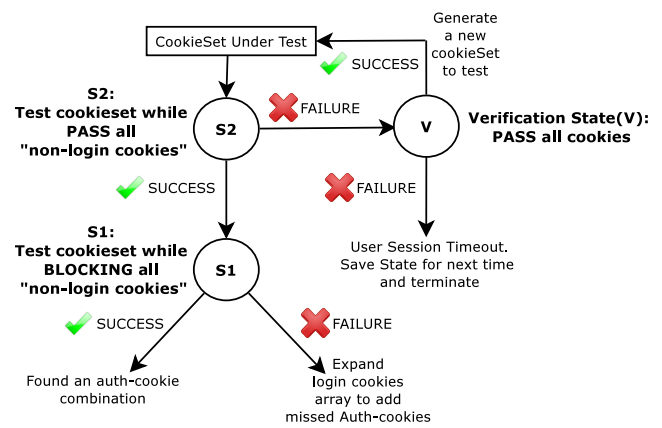


Figure 6: In contrast to Figure-5, while testing all current tops (coming DOWN in the POSET), a negative test result occurs when user is not logged in (expected result with most of the cookies blocked) where as a positive test result is when a user is still logged in.

"non-login cookies" similar to the one shown in Figure 4 to find the additional auth-cookies.

5 Implementation

We describe the implementation of Newton and an evaluation of its performance in terms of the page fetches, time, and overall download data required to compute auth-cookies for various sites.

5.1 Prototype: Chrome Extension

We implemented Newton as a Chrome browser extension. Our code is open source. Except for the scrapers of site-specific content, all of the code was written in JavaScript; we wrote the scrapers in a domain specific language.

We released a version of Newton to 12 users which were recruited by word of mouth advertisement in our university. We only allowed Newton to compute auth-cookie combinations on a pre-approved list of domains, with the users' consent. Many of the sites that we tested during deployment were not previously tested in the lab. None of these users have reported any performance problems or disruptions as a result of running Newton. As our experience with this initial deployment has been uniformly positive, we plan to release Newton to a wider audience, who we imagine may include both application developers, security-conscious users, and penetration testers.

Figure 7 shows an example of a scraper for `dropbox.com`. The code instructs the browser to go to the settings page for the user (line 3) and locate the HTML tags that correspond to the first and last name of the user (lines 5–14); it then stores the information to local storage (lines 16–21). We have shown one of the simpler examples of a scraper for simplicity; more examples are available on Github [22].

5.2 Performance Evaluation

To evaluate the feasibility of running Newton in real-time (e.g., to help users identify threats as they are browsing) as a crowd sourcing platform, we analyzed the time to compute auth-cookies, as well as the bandwidth overhead and number of page fetches required to compute the auth-cookies. We measured the performance of Newton using data from both our field deployment with alpha users and from our own tests, across 149 different websites. Of these 149 sites, 98 were English sites from the Alexa top-200 ranking, where we were able to create a user account. We tested the rest either because our pilot users used them, or because the site themselves

```

<div name=''dropbox''>
<action type=''fetch-url''>
https://www.dropbox.com/account#settings </action>
<div name=''first-name'' can_be_a_null=''no''>
<action type=''store'' field_type=''editable''>
td:contains(''First name'')+>input
</action>
</div>

<div name=''last-name'' can_be_a_null=''no''>
<action type=''store'' field_type=''editable''>
td:contains(''Last name'')+>input
</action>
</div>

<div name=''email'' can_be_a_null=''no''>
<action type=''store''
jquery_filter=''remove-children''>
td:contains(''Email'')+
</action>
</div>
</div>

```

Figure 7: Example of domain specific language for finding a user’s full name on Dropbox. For sites where we were able to create dummy accounts, we could write modules to infer the user’s username or full name from the site, which would then allow Newton to determine if a user was logged in.

were implemented using one of the popular eCommerce frameworks. We conducted these performance tests on a residential network in the United States with a downstream throughput of about 6–7 Mbps. The complete list of sites and gathered results of our analysis are available in a Google document [19]. We focused on testing sites that were either in the Alexa Top 200; popular among our alpha users; have both HTTP and HTTPS content and stored sensitive personal data; or were built using one of the frameworks that we tested.

Figure 8a shows a distribution of the number of page fetches that are required to compute the auth-cookies for a particular site. Newton can compute auth-cookies for 90% of sites with fewer than 100 page fetches; the median number of page fetches to compute auth-cookies for a site is 25. Figure 8b shows that about 80% of these computations require less than two minutes to complete, and Figure 8c shows that about 80% of these computations require the client to download less than 20 MB of data from the webpage for which Newton is performing the auth-cookie computation. These results show that Newton is efficient enough to operate in practice.

Another possible concern is that Newton might introduce excessive server load. As shown in Figure 8a, Newton identifies auth-cookies for most sites with fewer than 100 HTTP requests. We also note that the results of Newton’s auth-cookie computations are a one time effort for that part of the site. Additionally, auth-cookies are not specific to each user, so if Newton shares the results of these computations across users, users can avoid redundant computation.

6 Case Studies

It can be difficult for us to determine whether a site follows the guidelines [24], since in some cases, the tests require Newton to infer when the user logs into (or out of) a site, changes a password, or performs repeated multiple logins from the same account. We use various heuristics to infer when some of these events take place, including recording when a user provides input to a password box or clicks on logout links, storing encrypted passwords for later comparisons, and storing copies of cookies that were set during previous sessions. These heuristics are imperfect and will not work

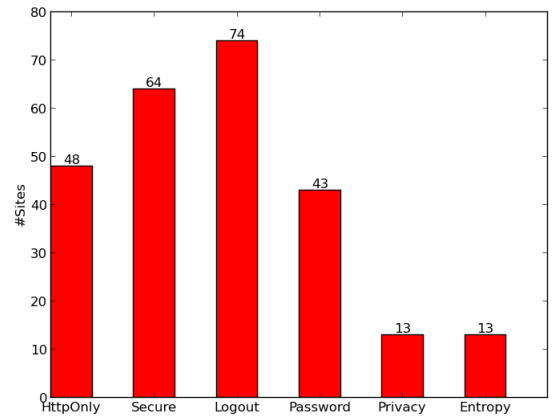


Figure 9: Results from our pilot study. “HttpOnly” indicates sites that failed to set that flag. “Secure” indicates sites that are susceptible to MITM attack. “Logout” indicates sites that did not invalidate auth-cookies post logout at server side. “Password” indicates sites that did not invalidate auth-cookies post password change. “Privacy” indicates sites that did not invalidate auth-cookies post logout at browser. “Entropy” indicates sites that did not change value of auth-cookies across sessions

Site	Fixed?	Notes
Yahoo	✗	Even if a user always accesses Yahoo over HTTPS, search history, notes, and stock list can be accessed by attacker. Yahoo told us that due to code complexity, they require a major overhaul to fix this issue and will not be doing so currently.
Vimeo	✓	Even if a user always accesses Vimeo over HTTPS, his entire account can be taken over. Vimeo fixed this after we notified them.
Magento (~250K sites)	?	Magento developers did not specify if they are working on a fix
WooCommerce (~650K sites)	✗	WooCommerce developers blamed the vulnerability on the underlying WordPress framework.
BigCommerce (~50K sites)	✓	We commend BigCommerce developers on speedy verification and pushing a fix in the production version.
Amazon	In progress	A cross-site scripting attack can result in session hijacking. An attacker can access user’s account completely. Amazon responded that they are aware of this bug and are working on a fix.

Table 1: Sites that acknowledged bugs discovered with Newton.

on all sites, but they work well enough for us to determine that many major sites are not following these best practices today. In Figure 9, the red bars show sites that violate the corresponding best practice. For each case, it was first necessary to discover which auth-cookies granted access to the website. To do so, we used Newton to first derive auth-cookies as a DNF formula, as mentioned in Section-2.1. In each case, we manually verified that the DNF that Newton derived was both complete and minimal. The rest of this section explores specific cases where we found a site violating a best practice, using these DNFs as the basis for our analysis. Testing for session fixation attacks requires another valid attacker session running, which is more difficult to automate.

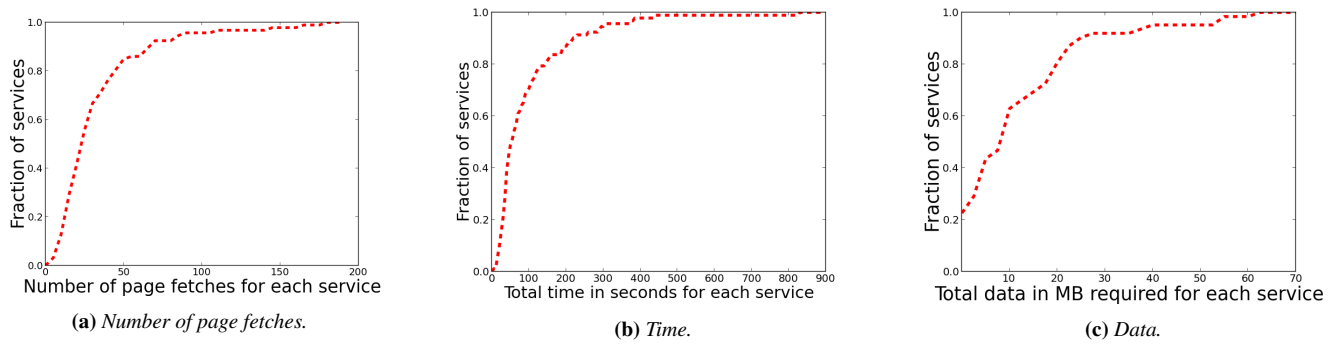


Figure 8: Page fetches, time, and data required to compute auth-cookies on Alexa top sites. In spite of our sub-optimal DNF terms generation algorithm, 90% of sites with fewer than 100 page fetches; 80% of these computations require less than two minutes to complete; and 80% require the client to download less than 20 MB of data.

6.1 Secure Against On-Path Attackers

We found that 64 out of 149 sites served at least some content over HTTPS, and yet had auth-cookies combinations that were unprotected, and would thus be vulnerable to on-path attackers (e.g., at a compromised WiFi hotspot). On all 64 sites, either the HSTS policy was absent or was incorrectly implemented. Our more detailed evaluation provides information about HSTS policy for each site [19]. In general, when a site’s contents are served over *HTTPS only* (which was the case for fourteen sites where we found vulnerabilities), not setting the `secure` flag indicates developer oversight. When a site serves content over HTTP and HTTPS, vulnerabilities may exist because web programmers have difficulty understanding how different auth-cookie combinations grant control to different parts of a website.

We present two examples of high-profile websites that have used the `secure` flag incorrectly with auth-cookies. In both cases it is interesting to note that the developer used the `secure` flag—indicating that the developer was aware of the need to secure certain cookies—but still did so incorrectly, resulting in vulnerabilities such as those described in Section 2.

Yahoo. Newton found that Yahoo Mail had auth-cookies Y AND T AND SSL and Yahoo Search had auth cookies Y AND T. Although both of these services serve content over HTTPS, only the SSL cookie is protected using the `secure` flag. This protects Yahoo Mail from cookie theft but leaves other services, such as search, vulnerable.

FlipKart. FlipKart is the largest eCommerce retail site in India with annual revenue of more than a billion dollars. Newton discovered that FlipKart has the following two auth-cookie combinations that grants access to a user’s private account over HTTPS: (VID AND NSID)OR SN None of these auth-cookies have the `secure` flag set, rendering both combinations vulnerable to theft.

GoDaddy. Until recently, we observed that GoDaddy had three different auth-cookie combinations that allowed access to a user’s personal account. Out of these three, only one combination was correctly protected. We recently tested GoDaddy again and found that they have independently fixed this issue, securing all three auth-cookie combinations. This behavior shows that developers understand the threat of auth-cookie theft and are willing to fix it once it is discovered. Tools such as Newton are vital for automating

this process by revealing such issues at development time through black-box testing.

Cost of implementing this recommendation: Setting the `secure` flag for auth-cookies of sites that are accessed entirely over HTTPS is easy. For sites that offer mixed content, developers must ensure that the auth-cookie combinations required for parts of the site served over HTTPS are mutually exclusive from those required for HTTP parts of the site. After that, it is necessary to set the `secure` flag for at least one auth-cookie from each auth-cookie combinations that allows access to the HTTPS parts. As we discussed in Section 2.3, an alternate way to tackle this problem is using HSTS, although deploying HSTS has proved to be difficult in practice.

To verify that this defense actually works in Newton, we implemented this attack and tested whether Newton could prevent it. We first verified that the Yahoo! Search’s auth-cookies Y and T can be stolen by manipulating a third-party website even if Yahoo! is always accessed over HTTPS. We then harvested the user’s search history using the stolen auth-cookies that we computed with Newton. Newton then forcefully sets the `secure` flag on one of them, at which point we verified that we could no longer execute the attack. (We only tested the runtime defense on the Yahoo Search site, but a similar runtime defense should be effective on other sites, as well. Similarly, if Newton detects that none of the auth-cookies in a particular combination have the `HttpOnly` flag set, it will force this flag on one of the auth-cookies to make the entire combination robust against XSS attack.) As mentioned in section-2.3, forcing flags on auth-cookies can break site functionality. However, Newton chooses minimal disruptions by only forcing flags on only one auth-cookie in each insecure auth-cookies set. Despite this, we are aware that such security enforcement can still break the site functionality. However, we leave the choice between security vurses functionality to user’s discretion.

6.2 Secure Against Malicious JavaScript

One mechanism that protects against the theft of auth-cookies using XSS is to set the `HttpOnly` flag on at least one auth-cookie in each auth-cookie combination, but there are legitimate reasons why a web developer may not set this flag. For example, some JavaScript programs may need to access an auth-cookie from the browser. To secure auth-cookies, however, the programmer should always have one auth-cookie in each combination for which the `HttpOnly` flag is set to protect against XSS exploits and JavaScript from malicious third parties.

Newton found 48 out of 149 sites that serve at least part of the contents or all of the contents over HTTPS (suggesting that the content and the cookies should be considered sensitive), yet did not set the `HttpOnly` flag to protect all auth-cookie combinations. Of these nine vulnerable sites, it appears that five of these vulnerabilities result from simple programming errors, since for those sites, *none* of the cookies had the `HttpOnly` flag set. We explore one such use case below.

Amazon. We found that Amazon AWS has the following auth-cookie combination: (`aws-at-main` AND `aws-userInfo` AND `aws-creds` AND `aws-x-main`) OR (`aws-at-main` AND `aws-userInfo` AND `aws-creds` AND `aws-ubid-main` AND `aws-session-id`). In this case, both auth-cookie combinations are protected because `aws-creds` has `HttpOnly` set to true. Yet, combinations to access a user's private account on Amazon's eCommerce site are as follows: (`at-main` AND `sess-at-main` AND `ubid-main` AND `x-main`) OR (`at-main` AND `sess-at-main` AND `ubid-main` AND `session-id`) None of these cookies have the `HttpOnly` attribute set, leaving authentication to Amazon's eCommerce site vulnerable to XSS attacks.

Cost of implementing this recommendation: A site operator can ensure that setting `HttpOnly` would not break site functionality by searching over the site's JavaScript code to determine whether any JavaScript accesses the `HttpOnly` auth-cookie. In cases where all auth-cookies are accessed by JavaScript code, the operator can secure the site by adding one more required auth-cookie to each combination that has the `HttpOnly` attribute.

6.3 Change Cookies Across Sessions

A capable attacker may be able to guess an auth-cookie combination if the cookie values have low entropy. Additionally, if the auth-cookie values remain unchanged across different sessions, then a single theft of auth-cookies could enable an attacker to continue authenticating to a web service as the user for an indefinite amount of time. Thus, at least one auth-cookie from each combination should change its value across the user's login sessions, and that cookie should also have sufficient entropy to make guessing hard.

We found that 13 out of 149 websites did not change client auth-cookies across sessions. Eight of these sites also send auth-cookies in HTTP cleartext and are thus vulnerable to other attack vectors. We found Twitter and GoDaddy to be particularly vulnerable:

Twitter. Twitter has only one auth-cookie combination with a single auth-cookie: `auth.token`. The value of this cookie is 40 hex characters that is constant across sessions. Also, because of this, if an attacker steals this cookie, then she can login to a victim's account even if the victim is not logged in.

GoDaddy. In the case of GoDaddy, we observed that only `auth_idp` changes in its value across sessions. All other auth-cookies retain their value across sessions, meaning that even though `auth_idp` changes, other static auth-cookies could be used to re-authenticate the user even after a session has expired.

Some web applications change auth-cookies across different sessions, but the entropy of these changes is low. For example, Facebook has two auth-cookies, `c_user` & `xs`. The `c_user` cookie is a user ID that does not change. The `xs` cookie appears to change across sessions; this cookie has 36 characters and multiple components, as follows: `119:ZHA1W3C7c7aBar:2:1406528127:6694`. Only the second part of this string is truly random; the other components

are guessable, as they refer to characteristics such as the time(*e.g.* 4th and 5th parts) when the user logs in.

Cost of implementing this recommendation: The wide variety of cryptographic libraries in different web languages make it possible to use cryptographic primitives to introduce more entropy into session cookies; the challenge, of course, lies in selecting good inputs to these functions that serve as reasonable sources of entropy that are difficult to guess [26].

6.4 Invalidate Cookies Upon Logout

When a user logs out, at least one auth-cookie from each set used for that session should be invalidated; otherwise, an attacker could masquerade as a user simply by stealing a user's auth-cookie set. Newton found 74 out of 149 sites where past session cookies allowed an attacker access to an account where the user was logged out. Additionally, upon logout, all auth-cookies should be deleted from the user's browser; at the very least, their values should be reset. Otherwise, a sufficiently long auth-cookie can still allow the webserver to track a user's activities, even after a user logs out!

Newton found thirteen sites that do not delete their auth-cookies from a user's browser after the user has logged out, including Amazon, as described in the example below:

Amazon. After a user logs out of Amazon, both `session-id` and `ubid-main` retain their values from the logged in session. Because each of them is a 17-digit number, each of them provides enough entropy to identify each user. (`at-main` AND `sess-at-main` AND `ubid-main` AND `x-main`) OR (`at-main` AND `sess-at-main` AND `ubid-main` AND `session-id`).

Cost of implementing this recommendation: The difficulty of implementing this recommendation depends on the complexity of the website. Invalidating a user's auth-cookies upon logout requires deleting the state of a user's past session. The growing popularity of NoSQL databases that provide only eventual (rather than strict) consistency, however, means that implementing this feature correctly would require some care in these cases. Deleting all auth-cookies from a user's browser should be simple, though. All it requires is identifying auth-cookies and resetting or deleting them upon user logout. A large site such as Amazon may require a careful analysis of the auth-cookie before simply deleting all cookies.

6.5 Let the User Control Auth-Cookies

An attacker can also steal auth-cookies by stealing a physical device; in such a case, a user should still be able to remotely invalidate his or her auth-cookies on the lost device. For this reason, we recommend that at least one auth-cookie in each combination should be derived from a user's password in some cryptographic way such as HMAC. With this rule, even if a user's password was leaked to an attacker, and if she is using it to login to victim's account, as soon as the victim realizes this and changes his password, all the auth-cookie combinations currently in use by attacker's session immediately become invalid.

On 43 out of 149 sites (including Monster, Mailchimp, Quora, OKCupid, GoDaddy, Comcast, and Amazon), we obtained access to a user's account after changing the password, using old auth-cookies that were generated in a past session which was established after logging in using an older password. (We performed our tests of stale auth-cookies approximately ten minutes after changing the user password; some of these sites may invalidate auth-cookies on a slower timescale.) This vulnerability means that if a user's auth-cookies are compromised, an attacker can retain access to the user's

account *even after the user discovers a compromise and changes his password!*

Cost of implementing this recommendation: Existing websites already typically store user passwords as a cryptographic hash. This existing input could in turn be used to derive an auth-cookie that changes whenever the user's password changes.

7 Related Work

In this section, we survey related work on web cookies. We survey related work in analyzing and computing auth-cookies for popular websites. There is extensive work in studying the use of auth cookies for user tracking (*e.g.*, [7, 25, 32]); our work is distinct from this body of work, as we focus on whether the cookies that are set by the primary domain for authentication are implemented correctly. We survey some of this related research below.

Identifying Auth-Cookies. Previous work focuses on identifying auth-cookies and checking if `HttpOnly` & `Secure` flags are set correctly. These approaches generally discover feature sets that distinguishes auth-cookies from other cookies, train the cookie-identification algorithm using these features, and possibly attempt to guess the auth-cookies for a site based on this inference. The most closely related work to Newton is the contemporary work from Calzavara *et al.* [8]. Using Selenium framework [27], the study found auth-cookies for popular sites by running tests in a controlled environment. Using this ground truth auth-cookies dataset, they identify a distinguishing feature set and train their algorithm to guess auth-cookies for unknown sites.

Newton is more significantly thorough than the algorithm from Calzavara *et al.*, because it accounts for the fact that different parts of the same site can be controlled by different auth-cookie combinations and DNF equations. For example, the DNF for Google Search comprises auth-cookies `HSID`, `SSID`, `SID`, but the DNF to access Google Mail requires `GX`, `LSID`, `APISID`, `SAPISID`, and the DNF for Google Calendar requires auth-cookie `CAL` in addition to all the auth-cookies required by Google Mail. Calzavara *et al.*'s algorithm appears to be incomplete, as their study discovered only `HSID`, `SSID`, `SID` as auth-cookies for Google, regardless of the portion of the site. The study also marked `APISID`, `SAPISID` as not being auth-cookies, even though Newton discovered that these cookies are in fact used for authentications on different parts of the Google site. The errors that Newton has uncovered in this previous analysis underscores the difficulty in computing the auth-cookies for each part of a website without performing a more exhaustive analysis. To allow for more exhaustive testing across a far wider diversity of sites and a larger user base, we developed Newton as a Chrome extension that can be deployed by anyone to compute auth-cookies in the field, which also allows it to crowd-source these measurements on a much larger scale than has been possible in any previous study.

Related work such as Sessionshield [20], Serene [9], CookiExt [6], and ZAN [30] attempt to find auth-cookies using features such as name, value length, and entropy, yet our analysis results demonstrate that none of these features are sufficient to completely identify auth-cookies.

Serene [9] also protects the sites against session fixation attacks. More recently, Zheng *et al.* [34] found that some of the top ranking sites do not implement cookies based authentication properly and are susceptible to injection attacks. While this work focuses on vulnerabilities causing cookie thefts due to the authentication mechanism lacking confidentiality property, these complementary

studies shows that sites often fail in asserting the integrity property as well.

Protecting Auth-Cookies. Newton also uses a different approach for protecting cookies in the user's browser. Whereas Calzavara *et al.* use the trained algorithm to guess auth-cookies and then possibly enforce the correct protection flags on them, doing so can miss important information and may also end up breaking the site functionality for a user. To understand this, suppose that a site has DNF: $(A \ \& \ B \ \& \ C) \text{ OR } (A \ \& \ B \ \& \ D)$ and that cookies C and D are protected with `HTTPOnly` and `Secure` flags. If their algorithm concludes that the auth-cookie DNF is $A \ \& \ B$ and enforces `HTTPOnly` and `Secure` flags, then that is unnecessary (since both terms in the DNF are already protected) and might end up breaking the site functionality if cookies A & B need to be accessed by Javascript or over plaintext HTTP.

Developing Security Best Practices for Auth-Cookies. More than 13 years ago, Fu *et al.* examined the use of cookies for client authentication on the web and uncovered various vulnerabilities; since this time, of course, the web has become increasingly more complex [11]. At that time, only single authentication cookies were used; although many of the recommendations from that work are now commonplace, the rise of complex web applications and authentication mechanisms begs for a re-appraisal of client authentication. The Open Web Application Security Project (OWASP) has set security guidelines that recognize the importance of this problem and also acknowledge that simply setting `secure` and `HttpOnly` flags on cookies is not a viable means of securing auth-cookies for many web applications [24]. However, even those guidelines are somewhat outdated and imprecise due to the fast pace at which the world wide web evolves. To the best of our knowledge, ours is the first work that makes these guidelines actually usable and provides a semi-automated tool to do so.

8 Conclusion

Web application programmers need better tools to evaluate the security of the cookies that are used to authenticate users, as well as recommendations for best practices for securing these cookies. Towards these goals, we have developed Newton, a tool that can discover all auth-cookie combinations that permit a user to access to a particular part or sub-service (*e.g.*, Amazon Web Services vs. Amazon Shopping, Google Mail vs. YouTube) of *any* site. Our analysis of 149 popular websites revealed security vulnerabilities in auth-cookie combinations on 65 sites that could be exploited by relatively weak attackers. Many of these sites acknowledged and even fixed the bugs that Newton discovered. As web authentication continues to become more complex, tools like Newton will be increasingly valuable to users, web programmers, and website administrators. We are planning a public release of Newton. Although we have designed Newton mainly as a tool for developers and penetration testers, Newton operates as an unobtrusive Chrome extension, so ultimately even security-savvy users can use it to test the security of sites that they regularly visit. Ultimately, crowdsourced data about auth-cookie mechanisms may help both users and web developers quickly identify and eradicate vulnerabilities in auth-cookies.

Acknowledgments

This work was funded by NSF Award CNS-1409635. We thank Arvind Narayanan, Saikat Guha, Srikanth Kandula, and Dave Levin for comments and feedback.

References

- [1] R. Agrawal and R. Srikant. Fast algorithms for mining association rules in large databases. In *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, pages 487–499, San Francisco, CA, USA, 1994. Morgan Kaufmann Publishers Inc. (Cited on page 4.)
- [2] Unsafe cookies leave WordPress accounts open to hijacking, 2-factor bypass. Ars Technica. <http://goo.gl/B1qLF7>, May 2014. (Cited on page 1.)
- [3] D. Atkins and R. Austein. *Threat Analysis of the Domain Name System (DNS)*. Internet Engineering Task Force, Aug. 2004. RFC 3833. (Cited on page 2.)
- [4] A. Barth. *The Web Origin Concept*. Internet Engineering Task Force, Dec. 2011. RFC 6454. (Cited on page 3.)
- [5] Bigcommerce: Ecommerce Software & Shopping Cart. <https://www.bigcommerce.com/>, Retrieved Feb. 2015. (Cited on page 1.)
- [6] M. Bugliesi, S. Calzavara, R. Focardi, and W. Khan. Automatic and robust client-side protection for cookie-based sessions. In *International Symposium on Engineering Secure Software and Systems. ESSoS'14*, 2014. (Cited on page 10.)
- [7] E. Butler. A Firefox extension that demonstrates HTTP session hijacking attacks. <http://codebutler.github.io/firesheep/>, 2010. (Cited on page 10.)
- [8] S. Calzavara, G. Tolomei, M. Bugliesi, and S. Orlando. Quite a mess in my cookie jar!: Leveraging machine learning to protect web authentication. In *Proceedings of the 23rd International Conference on World Wide Web*, 2014. (Cited on pages 2 and 10.)
- [9] P. De Ryck, N. Nikiforakis, L. Desmet, F. Piessens, and W. Joosen. Serene: Self-reliant client-side protection against session fixation. In *Proceedings of the 12th IFIP WG 6.1 International Conference on Distributed Applications and Interoperable Systems, DAIS'12*, pages 59–72, Berlin, Heidelberg, 2012. Springer-Verlag. (Cited on page 10.)
- [10] R. Fielding, J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, and T. Berners-Lee. *Hypertext Transfer Protocol – HTTP/1.1*. Internet Engineering Task Force, June 1999. RFC 2616. (Cited on page 4.)
- [11] K. Fu, E. Sit, K. Smith, and N. Feamster. Dos and don'ts of client authentication on the Web. In *Proc. 10th USENIX Security Symposium*, Washington, DC, Aug. 2001. (Cited on page 10.)
- [12] Hamster. <https://github.com/robertdavidgraham/hamster>, Retrieved Feb. 2015. (Cited on page 2.)
- [13] J. Hodges, C. Jackson, and A. Barth. *HTTP Strict Transport Security (HSTS)*. Internet Engineering Task Force, Nov. 2012. RFC 6797. (Cited on page 3.)
- [14] HttpOnly. <https://www.owasp.org/index.php/HttpOnly>, Retrieved Feb. 2015. (Cited on page 3.)
- [15] C. Jackson and A. Barth. Forcehttps: Protecting high-security web sites from network attacks. In *Proceedings of the 17th International Conference on World Wide Web, WWW '08*, pages 525–534, New York, NY, USA, 2008. ACM. (Cited on page 3.)
- [16] M. Kranch and J. Bonneau. Upgrading https in midair: Hsts and key pinning in practice. In *NDSS '15: The 2015 Network and Distributed System Security Symposium*, February 2015. (Cited on page 3.)
- [17] Magento: Ecommerce Software & Ecommerce Platform. <http://magento.com/>, Retrieved Feb. 2015. (Cited on page 1.)
- [18] Nbttool. <https://wiki.skullsecurity.org/Nbttool>. (Cited on page 2.)
- [19] Newton: Detailed Evaluation. <https://goo.gl/Rj7Vvw>. (Cited on pages 7 and 8.)
- [20] N. Nikiforakis, W. Meert, Y. Younan, M. Johns, and W. Joosen. Sessionshield: Lightweight protection against session hijacking. In *Proceedings of the Third International Conference on Engineering Secure Software and Systems, ESSoS'11*, pages 87–100, Berlin, Heidelberg, 2011. Springer-Verlag. (Cited on page 10.)
- [21] OWASP. OWASP Top 10 Application Security Risks - 2010. https://www.owasp.org/index.php/Top_10_2010-Main, 2010. (Cited on page 2.)
- [22] Appu: FPI examples. <https://goo.gl/qcllbR>. (Cited on page 6.)
- [23] OWASP: Double Submit Cookies. <http://goo.gl/qmW7o5>, Retrieved Feb. 2015. (Cited on page 3.)
- [24] Testing for cookies attributes. [https://www.owasp.org/index.php/Testing_for_cookies_attributes_\(OWASP-SM-002\)](https://www.owasp.org/index.php/Testing_for_cookies_attributes_(OWASP-SM-002)), Retrieved Feb. 2015. (Cited on pages 7 and 10.)
- [25] F. Roesner, T. Kohno, and D. Wetherall. Detecting and defending against third-party tracking on the web. In *9th USENIX Symposium on Networked Systems Design and Implementation. NSDI 2012*, 2012. (Cited on page 10.)
- [26] Seclist Advisory: Weak RNG in PHP session ID generation leads to session hijacking. <http://seclists.org/fulldisclosure/2010/Mar/519>, Retrieved Feb. 2015. (Cited on page 9.)
- [27] Selenium Web application testing system. <http://www.seleniumhq.org>. (Cited on page 10.)
- [28] Session hijacking attack. https://www.owasp.org/index.php/Session_hijacking_attack, Retrieved Feb. 2015. (Cited on page 1.)
- [29] SSL Pulse: Survey of the SSL Implementation of the Most Popular Web Sites. <https://www.trustworthyinternet.org/ssl-pulse/>, Retrieved on Nov, 2014. (Cited on page 3.)
- [30] S. Tang, N. Dautenhahn, and S. T. King. Fortifying web-based applications automatically. In *Proceedings of the 18th ACM Conference on Computer and Communications Security, CCS '11*, pages 615–626, New York, NY, USA, 2011. ACM. (Cited on page 10.)
- [31] Volusion Ecommerce Software & Shopping Cart Solutions. <http://www.volusion.com/>, Retrieved Feb. 2015. (Cited on page 1.)
- [32] R. J. Walls, S. S. Clark, and B. N. Levine. Functional privacy or why cookies are better with milk. In *Proceedings of the 7th USENIX Workshop on Hot Topics in Security, HotSec '12*, Bellevue, WA, Aug. 2012. (Cited on page 10.)
- [33] WooCommerce - a free eCommerce toolkit for WordPress. <http://www.woothemes.com/woocommerce/>, Retrieved Feb. 2015. (Cited on page 1.)

[34] X. Zheng, J. Jiang, J. Liang, H. Duan, S. Chen, T. Wan, and N. Weaver. Cookies lack integrity: Real-world implications. In *Proceedings of the 24th USENIX Conference on Security*

Symposium, SEC'15, pages 707–721, Berkeley, CA, USA, 2015. USENIX Association. (Cited on page [10](#).)